

Numeric Promotion by Chained Visitors

Recovering operand types for generic arithmetic in a C++ interpreter

Hob Nilre & Bo C. Herlin

2026-09-28

Abstract

Binary arithmetic in an interpreter depends on two runtime value kinds, while ordinary C++ virtual dispatch selects through one receiver. Chained visitors recover both concrete operand types, allowing a generic arithmetic expression and overloaded factories to return a correctly typed wrapper. We compare this construction with an explicit promotion table and C++17 variant visitation under one arithmetic contract. Adding subtraction and a fourth numeric kind exposes the changes each design requires; compiler checks cover every ordered type pair before and after the extension. The comparison identifies when preserving a polymorphic value hierarchy justifies visitors, when variant storage simplifies extension, and when an explicit table makes language policy easier to inspect. Short C++ examples separate dispatch, arithmetic conversions, result boxing and ownership. The contribution is a worked design comparison with reproducible evidence and concrete selection criteria.

PDF created: 2026-09-30 18:55:03 UTC

Latest on GitHub: <https://github.com/hobnilre/cpp-promote>

1 A small bridge between runtime values and typed arithmetic

An interpreter may store an integer and a floating-point value through the same base interface. When it multiplies them, the result depends on both concrete value kinds. The interpreter needs the flexibility of runtime values, while the arithmetic expression benefits from the concrete types that C++ normally knows at compilation.

The construction examined here connects those two settings with successive visitors. The first identifies the left operand's type and carries it into the second visitor. The second identifies the right operand's type. At that point an ordinary function-template specialization has both types available, so an expression such as `a * b` can use their scalar conversions. An overloaded factory then chooses the result wrapper from the expression's static result type.

This is an application of established visitor and multiple-dispatch techniques. Its appeal is the composition: one generic arithmetic expression covers a family of mixed-type cases, the dispatch machinery can be reused for other operations, and the interpreter receives the same common result interface throughout. The engineering question is when that composition is a useful choice. We answer it with three implementations of the same small numeric subsystem and two concrete change requests: add subtraction, then add another numeric kind. These are isolated implementation studies, not a completed interpreter integration.

The three concrete wrappers are named `Int`, `Long` and `Double`; their payloads have C++ types `int`, `long` and `double`. All derive from `Number`. We use C++17 as the reference language for the discussion and short extensions. The original visitor construction also compiles in C++98 mode in the checks reported below.

2 The operation that makes the design useful

The central arithmetic policy is small. With the wrapper classes and the result factories declared, the multiplication template is:

```
template<class T1, class T2>
class MUL {
public:
    static Number* op(T1 a, T2 b) {
        return number(a * b);
    }
};
```

Each wrapper supplies one implicit conversion to its payload type. There is no wrapper-specific multiplication overload in this example. Once T1 and T2 are concrete, built-in arithmetic candidates can use those conversions. The usual arithmetic conversions determine the scalar result type; overload resolution then selects a factory. These are distinct language mechanisms, specified by [WG21 \(2017\)](#) in its treatment of [expressions and built-in operators](#).

The factories have the following bodies:

```
Number* number(int x)      { return new Int(x); }
Number* number(long x)     { return new Long(x); }
Number* number(double x)   { return new Double(x); }
```

For an `Int` multiplied by a `Double`, the scalar expression has type `double`, so the last factory is the exact match. For two `Int` operands it has type `int`, so the first is selected. A result numerically equal to seventeen does not select its wrapper by inspecting that value. Its type was already determined for the selected template specialization.

The wrappers returned through `Number*` remain dynamically distinct. The common interface hides their types from the caller until another operation visits them. Printing also passes through that interface. Consequently, identical printed numbers can conceal different result kinds: the supplied demonstration multiplies integer and floating representations of seventeen and prints four occurrences of 289, although only its integer-by-integer result is an `Int`.

3 Recovering the ordered operand pair

The common `Number` interface provides a virtual acceptance function. A `Visitor` has one virtual overload for each supported wrapper. Acceptance in a concrete wrapper selects the overload whose parameter is that wrapper type. Overload selection uses a static parameter type; virtual dispatch then selects the visitor implementation.

The binary entry point creates the visitor that will first process the left operand:

```
Number* mul(Number& a, Number& b) {
    return a.accept(VisitorLhs<VisitorRhs, MUL>(b));
}
```

The left visitor holds a reference to the right operand through `Number`. When the left operand is an `Int`, its corresponding overload has a concrete `Int` reference named `lhs`. The following statement is its complete forwarding action; here `V` denotes the right-visitor template and `OP` the operation template:

```
return rhs.accept(V<Int, OP>(lhs));
```

The new right visitor retains `lhs` as a reference whose type is now known. If the right operand is a `Double`, acceptance selects its `Double` overload. Inside the right visitor, instantiated with left type `T`, that overload ends with:

```
return OP<T, Double>::op(lhs, rhs);
```

The selected operation is therefore multiplication specialized for the ordered pair `(Int, Double)`. Reversing the input kinds selects `(Double, Int)`. The chain never swaps operands to put the apparently wider kind first. Multiplication conceals this distinction because it is commutative on the ordinary finite examples; subtraction exposes it immediately.

Figure 1 separates the two runtime selections from the arithmetic they reach. The compiler instantiates the relevant typed operations when the program is built. Runtime dispatch selects among those compiled paths; it does not instantiate a new template when an interpreter value arrives.

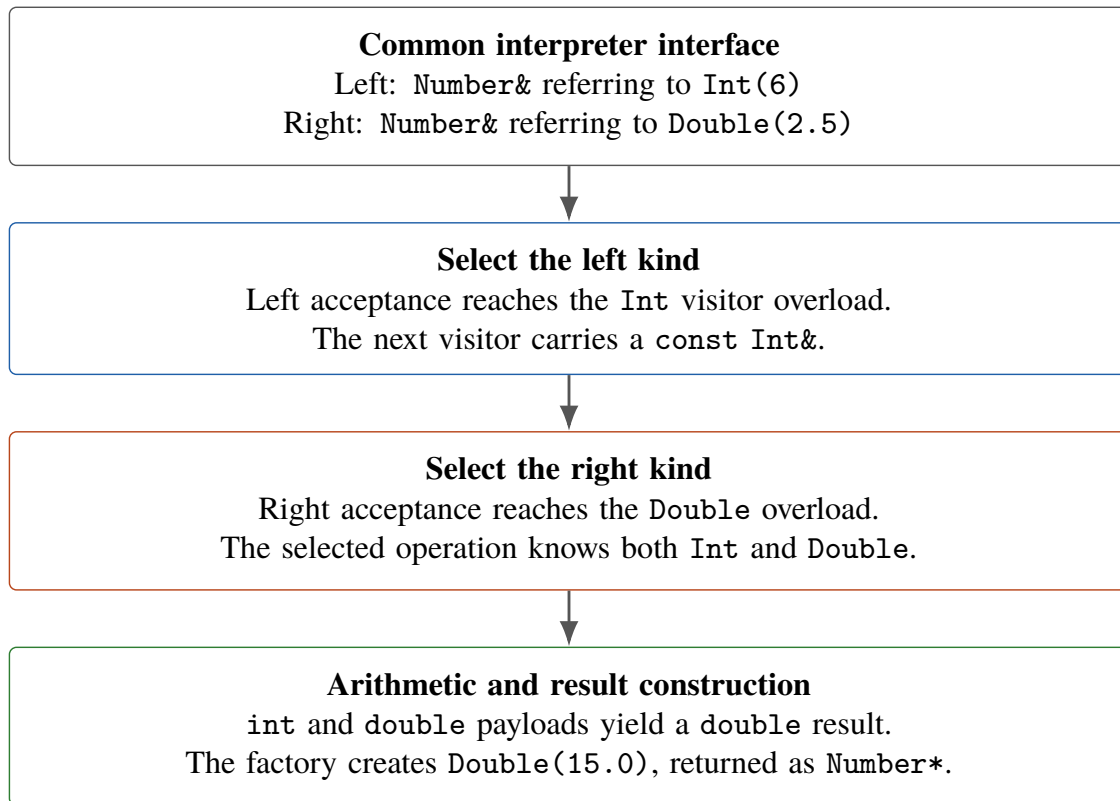


Figure 1. Two successive operand selections reach a typed operation. Schematic trace for an `Int` left operand and a `Double` right operand; the green result box is chosen from the scalar expression type.

There are two operand-selection stages but four virtual call sites on this source-level path: left acceptance, the left visitor overload, right acceptance, and the right visitor overload. A compiler may devirtualize or inline some calls. Calling this a two-stage visitor construction does not establish that a machine executes exactly two indirect calls.

4 Result types and what promotion means here

For multiplication in the declared family, the complete table is:

Table 1. Result wrapper for every supported ordered operand pair, assuming defined arithmetic and successful allocation.

Left wrapper / right wrapper	Int	Long	Double
Int	Int	Long	Double
Long	Long	Long	Double
Double	Double	Double	Double

Let $K = \{I, L, D\}$ denote these wrapper kinds, with scalar types $\tau_I = \text{int}$, $\tau_L = \text{long}$ and $\tau_D = \text{double}$. Write $P(i, j)$ for the table's result kind. Within this family, it selects the later kind in the order I, L, D . The visitors do not implement that rank rule themselves: it describes the arithmetic reached after dispatch. Introducing unsigned integers, different operators or user-defined arithmetic requires a fresh analysis.

The title uses promotion in the interpreter-design sense of choosing an arithmetic result kind. C++ uses *integral promotion* more narrowly; the mixed cases here rely on its wider set of usual arithmetic conversions. In particular, `long` outranks `int` even on an implementation where their widths happen to match. Moving to `double` also does not guarantee exact preservation of every integer value.

For a defined multiplication, let $c_{i,p}$ convert a payload of kind i to the scalar type associated with $p = P(i, j)$, and let b_p construct its wrapper. The intended numeric operation is

$$\mathcal{M}((i, x), (j, y)) = b_p(c_{i,p}(x) \times_{\tau_p} c_{j,p}(y)). \quad (1)$$

The symbol $\times_{\tau_p}$ means host arithmetic in that type, including its applicable finite-precision semantics. Equation (1) is defined only where conversions and arithmetic are defined; allocation must also succeed. The factory selection is exact for all scalar result types in this table.

For example, integer six times double two-and-a-half yields a `Double` containing fifteen; long seven times integer six yields a `Long` containing forty-two. Two integer sevens still produce an `Int`. The type choice does not inspect the magnitude of the answer or widen automatically when an integer calculation overflows.

The ordered selection has a short correctness argument. Suppose each operand is a supported wrapper, acceptance exposes that wrapper correctly, and both operands outlive the synchronous visitor calls. Then operands of kinds i and j reach the operation specialized for (i, j) with the original left and right values in that order.

The first acceptance selects the overload for kind i . That overload creates a right visitor parameterized by i and retaining the left operand. Acceptance of the right operand selects its kind- j overload. Its body invokes the specialization for (i, j) , passing the retained left operand first and the right operand second. These exhaust the two selections; there is no operand exchange or value-dependent branch in between.

When the selected arithmetic and result construction are well-defined, scalar conversion and exact factory selection then give (1). This separates dispatch correctness from arithmetic validity and from the correctness of the acceptance cast discussed below.

5 The interpreter chooses the arithmetic policy

The reuse boundary is the operation template. Replacing multiplication with subtraction leaves the visitor structure intact and preserves operand order. A division template using ordinary `/` inherits integer division for two integer payloads. Thus integer seven divided by integer two produces an `Int` containing three, while a double divisor produces a `Double` containing three-and-a-half.

An interpreter whose division always returns floating point can use another policy. The following example requires `<stdexcept>` and the wrappers' scalar accessor `get()`:

```
template<class A, class B>
struct REAL_DIV {
    static Number* op(const A& a, const B& b) {
        if (b.get() == 0)
            throw std::domain_error("zero divisor");
        return number(static_cast<double>(a.get()) /
                      static_cast<double>(b.get()));
    }
};
```

The same visitor chain can select this policy for any of the nine pairs. The explicit conversions give it a different result rule from the multiplication table; the zero check gives it an explicit failure case. The example still inherits the host's floating-point representation and rounding. Rational division or arbitrary-precision arithmetic would require their own payloads and policies.

Other operations reveal a second boundary: factory overloads can themselves convert a result. A comparison expression yields `bool`, but the three factories above select the integer overload, producing an `Int`. A float expression selects the double overload. Missing an exact factory therefore need not cause compilation to fail. If the guest language distinguishes Boolean values or requires exact result boxing, the factory interface must enforce that policy explicitly.

An operation can instead reject unsupported combinations. Such rejection must itself be a valid implementation for those typed pairs. Merely writing integer remainder generically is insufficient: the right-visitor overloads also form floating-point cases, for which built-in `%` is invalid. The compile-time rejection occurs even in the focused test whose actual call supplies two integers. Runtime usage alone does not remove the other combinations from this visitor implementation.

6 Three implementations of one arithmetic contract

6.1 Common inputs and expected results

The comparison starts with the same three payload types and C++ multiplication semantics. Both operand order and result kind must be preserved. The exact test inputs are integer six, long seven and double two-and-a-half; they avoid overflow and inexact arithmetic. Every implementation must return the nine kinds above and the following values:

Table 2. Independent expected multiplication values shared by all three implementations.

Left / right	int (6)	long (7)	double (2.5)
int (6)	36	42	15
long (7)	42	49	17.5
double (2.5)	15	17.5	6.25

The visitor implementation retains `Number` and its wrappers. The other two use `Value = std::variant<int, long, double>` as their storage container, requiring `<variant>`. This holds storage constant between the explicit-table and variant-visitation alternatives. In the explicit design, `variant` supplies storage and a tag only; its visitor is not used to implement arithmetic. Result wrappers are owned by `std::unique_ptr<Number>` in the checks; variant results own their scalar value directly. These lifetimes provide the same cleanup guarantee for the exercised calls, but allocation and layout differ. The comparison measures functional agreement and required source changes, not execution speed.

6.2 An explicit promotion table

An explicit implementation records the promotion rule separately from arithmetic. Here the enum values match the three storage indices:

```
enum Kind { I, L, D };
constexpr Kind promotion[3][3] = {
    {I, L, D}, {L, L, D}, {D, D, D}
};
```

The helper `as<T>(v)` switches on the active storage index, extracts the corresponding scalar with `std::get`, and converts it with `static_cast<T>`. It rejects a valueless variant. With that helper, `<stdexcept>`, and the `Value` alias above, the shared dispatcher is:

```
template<class Op>
Value explicit_apply(const Value& a, const Value& b, Op op) {
    if (a.valueless_by_exception() || b.valueless_by_exception())
        throw std::bad_variant_access();
    switch (promotion[a.index()][b.index()]) {
        case I: return op(as<int>(a), as<int>(b));
        case L: return op(as<long>(a), as<long>(b));
        case D: return op(as<double>(a), as<double>(b));
    }
    throw std::logic_error("invalid promotion kind");
}
```

For multiplication, supply the generic scalar policy:

```
auto multiply = [](auto x, auto y) { return x * y; };
```

The call is `explicit_apply(a, b, multiply)`. This factored baseline needs one arithmetic expression and three typed conversion paths, not nine handwritten multiplication bodies. The table has nine policy entries and is reusable for operations governed by the same common-type rule. Explicit tables can be a good fit when the guest language’s conversion policy must be independently inspected or changed.

Its distinction from the visitor is where type information is discarded. Before calling `op`, the explicit dispatcher converts both operands to the chosen common scalar type. The operation sees that type twice. The visitor retains the original ordered wrapper pair until the operation itself chooses conversions. A policy that distinguishes original operand kinds therefore needs extra information in the explicit design, such as the original tags; the common-type conversion alone is insufficient.

6.3 Variant visitation

Using the same `Value` storage, the corresponding reusable dispatcher is:

```
template<class Op>
Value variant_apply(const Value& a, const Value& b, Op op) {
    return std::visit([&](auto x, auto y) -> Value {
        return op(x, y);
    }, a, b);
}
```

The call `variant_apply(a, b, multiply)` reaches a specialization for the original ordered scalar pair. C++ chooses the product type, and construction of `Value` selects its exact matching alternative for these numeric results. The explicit `Value` return type gives every visitor specialization one return type, as required by [C++17 visitation](#). No promotion table is maintained.

This provides much of the same arithmetic convenience as the chained visitors, with less custom dispatch machinery in this example. Its representation is a closed union of scalar alternatives. Adopting it throughout an interpreter whose interfaces already pass `Number` objects is a separate representation change; it is not a replacement of just the multiplication body.

7 Two concrete extension requests

7.1 Add ordered subtraction

For the wrapper design, add the operation policy:

```
template<class A, class B>
struct SUB {
    static Number* op(const A& a, const B& b) {
        return number(a - b);
    }
};
```

Invoking `a.accept(VisitorLhs<VisitorRhs, SUB>(b))` uses the existing visitor interfaces, factories and wrappers unchanged. For either alternative dispatcher, supply:

```
auto subtract = [] (auto x, auto y) { return x - y; };
```

Both `explicit_apply(a, b, subtract)` and `variant_apply(a, b, subtract)` preserve left-minus-right. No table change is required because these three scalar types have the same common-type rule for multiplication and subtraction. In particular, integer six minus double two-and-a-half produces double 3.5; reversing the operands produces double -3.5. All nine ordered pairs are checked against independently written expected values and kinds.

All three factored designs therefore make this operation extension small. The visitor’s advantage is providing that reuse within the existing wrapper hierarchy. Adding a more demanding policy still requires handling every admitted type pair; generic syntax does not make floating remainder valid or turn host integer division into real division.

7.2 Add a fourth numeric kind

The second request adds `LongLong`, whose scalar payload is `long long`. This extension uses C++17, rather than claiming to extend the original C++98 demonstration unchanged. Call its kind Q . It is a distinct C++ type from `long` even on the tested platform where both have 64 bits. The resulting rule for multiplication and subtraction is:

Table 3. Four-kind result table. Q denotes the added long long kind; storage order is not promotion rank.

Left / right	I	L	D	Q
I	I	L	D	Q
L	L	L	D	Q
D	D	D	D	D
Q	Q	Q	D	Q

The actual implementation changes are:

Table 4. Changes to the three concrete implementations, excluding common test updates and interpreter facilities outside this numeric subsystem.

Design	Changes needed for <code>long long</code>
Chained visitors	Declare <code>LongLong</code> ; add its base visitor overload and forwarding case in both visitor stages; define its wrapper/conversion and exact factory.
Explicit table	Append the storage alternative and tag; add extraction/conversion and dispatch cases; expand the promotion table by seven entries.
Variant visitation	Append <code>long long</code> to <code>Value</code> ; retain <code>variant_apply</code> and both scalar policies unchanged.

The visitor’s new wrapper uses `NumberT<LongLong, long long>`, so its inherited acceptance exposes `LongLong` through the added overload. Merely subclassing an old wrapper and inheriting its acceptance would continue to dispatch as the old kind. The existing multiplication and subtraction templates need no new bodies: built-in conversions and the new exact factory handle their added pairs.

In both variant-backed implementations the storage list becomes:

```
using Value = std::variant<int, long, double, long long>;
```

Appending the new type keeps the old storage indices unchanged. Its index is now greater than `double`'s, although mixing the two must still produce `double`. Thus taking the maximum tag would fail; the explicit table must encode the rule, while visitation leaves it to typed C++ arithmetic.

With the added payload nine, `LongLong(9) * Long(7)` returns a `LongLong` containing 63, and `LongLong(9) * Double(2.5)` returns a `Double` containing 22.5. The sixteen ordered pairs are checked for both operations in all three implementations, including the original nine. Moving from three kinds to four adds seven semantic pairs per operation; generally the increase is $(n+1)^2 - n^2 = 2n + 1$. The variant design has fewer source edits here, but its generic operation must still be valid for the expanded set of pairs.

7.3 What the comparison establishes

These change requests support a concrete choice. Chained visitors suit an existing `Number` hierarchy when operations are added more often than value kinds and the operation should receive the original pair of types. Variant visitation is a compact choice for this small value family when its storage representation can be adopted. An explicit table makes a separately specified conversion policy visible and reusable; it need not duplicate arithmetic for every pair. A language with several operation-specific conversion rules must represent those distinctions in whichever design it uses.

The visitor construction does not eliminate the Cartesian product of supported kinds. There are n^2 possible binary pairs and n^k tuples for k operands. Templates factor the source description; these counts are neither emitted-code sizes nor runtime instruction counts. For comparison, the explicit design collapses pairs into common-type paths, provided that loss of original type identity is permitted by the arithmetic contract.

On the visitor's source-level path, a successful binary call has four virtual call sites, two temporary visitor objects, and one result allocation. The original multiplication copies two small wrappers into its by-value operation; the subtraction policy above uses references. With fixed-size payloads, dispatch depth is independent of the number of kinds. A speed comparison would require matched allocation, ownership, workloads and compiler settings; the functional comparison here establishes no speed ranking.

8 Conditions for a reliable implementation

8.1 Acceptance and object identity

The wrappers share a template base parameterized by the derived wrapper and scalar type, a form of the curiously recurring template pattern. Its acceptance body must expose the actual derived wrapper to the visitor. The original body uses a reinterpret cast for this step. That is not a portable general base-to-derived adjustment. For the intended public, unambiguous, nonvirtual base relationship, the appropriate expression inside the template base is:

```
return v.visit(*static_cast<const T*>(this));
```

This follows the standard's [base-to-derived static-cast rules](#). It still requires that the object contain the claimed derived wrapper; casting a standalone template-base instance does not make it one. A robust wrapper design should restrict base construction and copying so that this invariant is maintained, including against slicing. The private checks distinguish the unchanged example from a copy with this proposed repair.

A modified test places the template base after another polymorphic base. On the tested platform, the original reinterpret-cast version receives a sanitizer diagnostic, while the static-cast version passes the

same arithmetic checks. This illustrates the missing pointer adjustment. It is not a claim that the original single-inheritance demonstration failed on that platform, nor that testing proves every possible hierarchy valid.

8.2 Lifetimes, ownership and arithmetic boundaries

Both visitors hold references to operands. Their temporary visitor objects are used synchronously during the acceptance calls, and no reference is retained in the scalar result wrapper. Both operands must remain alive throughout that chain. Deferred or asynchronous visitation would need a different lifetime design.

The original factories transfer an owning raw pointer to the caller. The demonstration deletes each result, and `Number` has a virtual destructor. In an interpreter, an owning result handle such as `std::unique_ptr<Number>` makes cleanup explicit; propagating that ownership type through the visitor interface avoids relying on every caller to remember deletion. Allocation and policy exceptions also need the interpreter's error convention. The original public entry point takes non-const references, although the visitor path reads const operands and could expose a const-correct entry point.

Signed multiplication outside its representable range is not automatic widening. Negating the minimum representable signed integer is another invalid case for the unchanged unary policy. A checked arithmetic policy must detect these cases before evaluating the overflowing expression. This is an arithmetic requirement separate from recovering the two operand kinds.

Integer-to-floating conversion can lose precision. In the checked environment, `long` is 64 bits and `double` has 53 binary significand bits. Multiplying the long integer $2^{53} + 1$ by double one produces the observed double value 2^{53} . The input integer is not exactly representable in that floating format. The C++ conversion rule permits implementation-dependent rounding when an in-range integer is not exactly representable; these widths and this rounding observation are not universal C++ platform guarantees. See the working draft's [floating-integral conversions](#).

The scalar payloads in the supplied wrappers are immutable, and unary negation and printing each use the existing single-operand interface. They do not require the two-stage construction. Their distinct role helps keep the binary dispatch mechanism small.

9 Relationship to existing techniques

[Pirkelbauer, Solodkyy and Stroustrup \(2007\)](#) discuss Visitor as a workaround for multiple dispatch and develop open multimethods for C++. Their discussion also identifies the visitor interface's restriction on adding new subclasses. The present construction combines successive visits with templates to recover an ordered type pair inside a fixed value family. It does not implement their open-method extension or adopt its performance results.

The [Boost.Variant](#) documentation already provides an operation applied to the contents of two variants. C++17 standardizes [visitation of multiple variants](#). Recovering a type pair for generic code is therefore an established capability. The contribution of this case study is its explicit connection to numeric conversion and result boxing, together with implemented change requests that expose the different maintenance obligations. It supplies evidence for a design choice, rather than a new multiple-dispatch mechanism.

10 Verification and demonstrated scope

The unchanged demonstration was compiled in C++98 and C++17 modes with GCC 15.2.0 and Clang 20.1.8 on x86-64 Linux. Both produced its four printed values of 289. Focused C++17 harnesses then

checked dynamic wrapper kinds and exact small values for all nine multiplication pairs and all nine subtraction pairs. Checks also exercised ordinary and explicitly floating division, zero-divisor rejection in the latter, implicit factory conversions, unary negation and unchanged input payloads.

The same positive checks passed for the original and static-cast copies under address and undefined-behavior sanitizers. Separate negative checks diagnosed signed overflow and rejected a remainder policy lacking floating-point cases. The altered inheritance-layout probe has the distinct outcome described above.

The design comparison uses a private static-cast copy of the wrapper implementation and two separately implemented variant-backed alternatives. Both compilers checked all nine base-family pairs and all sixteen extended-family pairs for multiplication and subtraction in each design. Expected kinds and exact values were written independently of the dispatchers; agreement among implementations alone was not the oracle. The checks compiled the article’s actual C++ excerpts in their declared contexts and verified zero rejection for the floating division policy. Generated source differences retain the actual type-extension changes. These are finite implementation checks, not a proof of portability, arithmetic safety for all values, or comparative performance. The ordered-selection argument supplies the structural reasoning under its stated assumptions.

The workload contains exactly representable small values and exercises a fixed family of built-in scalar types. It does not establish equal behavior for every custom numeric type or every floating environment. Arbitrary precision, operation-specific rejection and value-dependent overflow widening would need additional policies and checks. Integration into parsing, storage management and the rest of an interpreter remains outside this study.

11 Conclusion

Chained visitors make a dynamically selected operand pair available to statically typed generic code. The first stage retains the left type, the second supplies the right type, and ordinary arithmetic plus overloaded result construction completes the operation. The nine-case table shows exactly what the compact multiplication expression achieves for `int`, `long` and `double`.

The worked comparison makes its useful scope concrete. All three designs add subtraction with a small policy change. Adding `long long` requires coordinated visitor edits, a table and dispatch extension in the explicit design, and only a storage-list change for the variant design’s arithmetic. Chained visitors earn their place when retaining a polymorphic value interface matters. Their strength is a reusable connection to typed operations with explicit conversion and boxing boundaries; the alternatives show when another representation or a separately maintained conversion policy is the better fit.

References

1. ISO/IEC JTC1/SC22/WG21 (2017). *Working Draft, Standard for Programming Language C++*. N4659, 21 March 2017. [Committee draft](#). Cited clauses: expressions, built-in operators, static casts, floating-integral conversions and variant visitation.
2. Pirkelbauer, P., Solodkyy, Y., and Stroustrup, B. (2007). Open multi-methods for C++. *Proceedings of the 6th International Conference on Generative Programming and Component Engineering*, 123–134. [doi:10.1145/1289971.1289993](#); [author manuscript](#).
3. Friedman, E., and Maman, I. *Boost.Variant: apply_visitor*. Boost 1.47.0 library documentation. [Versioned reference](#).